

Clase Práctica : Nivel de Transporte

Tomás F. Melli

October 2025

Índice

1	Introducción	2
1.1	Identificación de procesos: puertos y sockets	4
1.2	Clasificación de los protocolos de transporte	5
2	El protocolo UDP	5
2.1	Formato del datagrama UDP (según RFC 768)	6
2.2	Puertos como colas de mensajes	6
3	El Protocolo TCP	7
3.1	Diferencias básicas con UDP	7
3.2	Formato del segmento TCP	7
3.3	Esquema de reconocimiento	8
3.4	Modo de operación	8
3.5	Establecimiento de conexión: Three-Way Handshake	8
3.6	Apertura simultánea	9
3.7	Diagrama de estados: apertura	9
3.8	Cierre de conexión: Four-Way Handshake	10
3.9	Cierre simultáneo	10
3.10	Diagrama de estados: cierre	10
4	Ejercicio 1: análisis de una secuencia de segmentos TCP	11
5	Ejercicio 2: Captura de paquetes TCP con cierre de conexión	12
6	Port Scanning	13
6.1	Clasificación de puertos	13
6.2	SYN scan (o half-open scan)	13
6.3	Connect scan (o TCP scan)	14
7	Herramientas de análisis TCP/IP	14
7.1	Nmap	14
7.2	Telnet	15
7.3	Netstat	15
7.4	Netcat	16
8	Firewall	18
8.1	Tipos de Firewalls	19
9	NAT	19
9.1	NAT ilustrado	20
9.2	NAPT (Network Address and Port Translation)	21

1 Introducción

El modelo OSI (Open Systems Interconnection) define una arquitectura de comunicación en capas que permite la interoperabilidad entre diferentes sistemas y redes. Cada nivel se encarga de una función específica dentro del proceso de comunicación, estableciendo una jerarquía clara que facilita el diseño, la implementación y el diagnóstico de redes.

Nivel físico

El **nivel físico** se ocupa de la transmisión real de bits a través de un medio físico determinado. Define las características eléctricas, mecánicas y procedimentales necesarias para establecer, mantener y desactivar la conexión física entre dispositivos. En este nivel se encuentran aspectos como:

- Tipo de medio de transmisión (cobre, fibra óptica, radiofrecuencia).
- Codificación de los bits.
- Sincronización de señales.
- Topología física de la red.

Su objetivo fundamental es garantizar que un flujo de bits transmitidos por un extremo sea recibido correctamente en el otro, sin preocuparse por el contenido de la información.

Nivel de enlace de datos

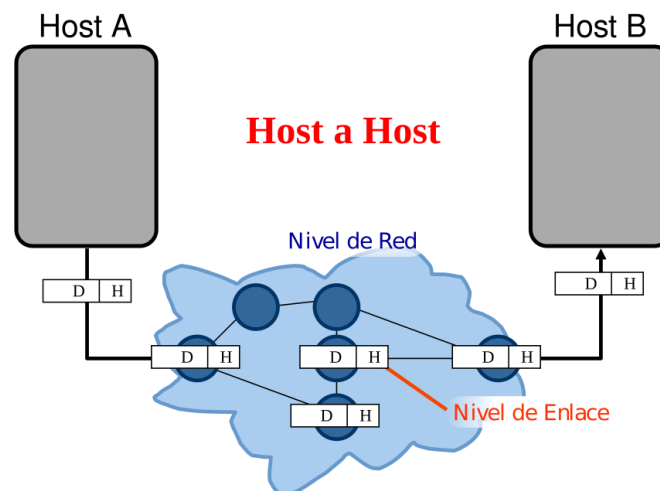
El **nivel de enlace** se encarga de conectar dispositivos dentro de una misma red local (LAN), proporcionando un canal libre de errores entre dos nodos directamente conectados. Su unidad básica de transmisión es la **trama** (*frame*), que incluye información de control de errores, direcciones físicas (MAC) y delimitadores de inicio y fin.

Las funciones principales de este nivel incluyen:

- Detección y corrección de errores.
- Control de flujo.
- Acceso al medio compartido (por ejemplo, CSMA/CD en Ethernet).
- Encapsulamiento y direccionamiento a nivel local.

En resumen, mientras el nivel físico transmite bits, el nivel de enlace se asegura de que esos bits lleguen correctamente entre nodos adyacentes.

Nivel de red



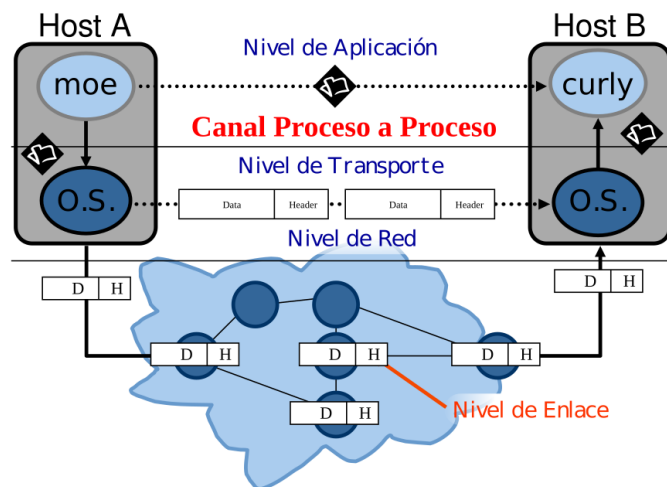
El **nivel de red** define cómo interconectar múltiples redes y dirigir el tráfico de paquetes entre sus destinos finales. Introduce el concepto de **direccionamiento lógico** (como las direcciones IP) y la **encaminación** (*routing*) de paquetes a través de diferentes nodos intermedios (routers).

Sus principales responsabilidades son:

- Encapsulamiento de datos en paquetes.
- Asignación y gestión de direcciones lógicas.
- Determinación de rutas óptimas.
- Fragmentación y reensamblado de paquetes cuando el tamaño máximo de transmisión (MTU) varía entre redes.

Este nivel implementa un servicio de *mejor esfuerzo* (*best effort*), lo que significa que no garantiza la entrega ni el orden de los mensajes, ni evita duplicados.

Nivel de transporte



Hasta este punto, el modelo OSI permite la comunicación **entre hosts**, pero las aplicaciones requieren un canal **entre procesos**, que garantice una comunicación confiable y ordenada. Aquí entra en juego el **nivel de transporte**, cuya misión es ofrecer a las aplicaciones un canal lógico extremo a extremo, libre de los problemas que presentan las capas inferiores.

Objetivo y funciones

El nivel de transporte implementa mecanismos para asegurar que los datos enviados por una aplicación en un host sean recibidos correctamente por la aplicación destino en otro host. Sus principales funciones son:

- Entrega confiable y ordenada de mensajes.
- Eliminación de duplicados.
- Segmentación y reensamblado de datos de tamaño arbitrario.
- Control de flujo, evitando la saturación del receptor.
- Multiplexación, permitiendo que múltiples procesos en un mismo host compartan la red.
- Corrección de errores mediante retransmisión o verificación de integridad.

En esencia, el nivel de transporte transforma un servicio de red no confiable en un canal confiable entre procesos.

Similitudes con el nivel de enlace

Tanto el nivel de transporte como el de enlace deben resolver problemas de control de errores, secuenciamiento y control de flujo. Sin embargo, existen diferencias importantes:

- El nivel de enlace opera sobre un canal físico entre nodos adyacentes.
- El nivel de transporte opera sobre una red completa, donde los paquetes pueden ser almacenados temporalmente, perderse o llegar fuera de orden.
- El transporte requiere direccionamiento explícito y puede implicar establecimiento de conexión, lo que lo hace conceptualmente más complejo.

1.1 Identificación de procesos: puertos y sockets

Para que los procesos puedan comunicarse, deben identificarse de manera única. No es posible usar **PIDs** (identificadores de proceso del sistema operativo), ya que tienen alcance local y cambian con el tiempo. En la arquitectura **TCP/IP**, se utilizan **puertos**, valores numéricos de 16 bits que, junto con la dirección IP, identifican de forma unívoca un proceso o servicio.

Los **sockets** combinan una dirección IP y un número de puerto, permitiendo establecer conexiones entre procesos de distintos hosts. Existen dos enfoques principales:

- **Puertos bien conocidos:** asignados a servicios estándar (por ejemplo, HTTP en el puerto 80, SMTP en el 25, SSH en el 22).
- **Port mappers:** procesos que mantienen una tabla de servicios activos y sus puertos (ejemplo: RPC).

En sistemas Linux, el archivo `/etc/services` contiene el listado de puertos asociados a los servicios conocidos; se utiliza para que, desde el código, una aplicación obtenga el puerto asociado a un nombre de servicio.

A modo de ejemplo, una sección típica muestra entradas como:

```
ftp-data  20/tcp
ftp       21/tcp
fsp       21/udp
ssh       22/tcp
ssh       22/udp
telnet    23/tcp
smtp      25/tcp
time      37/tcp
time      37/udp
fspd
mail
timserver
```

- **ftp-data (20/tcp):** Puerto usado por la conexión de datos del protocolo FTP (File Transfer Protocol). FTP establece una conexión de control (puerto 21) y una o más conexiones de datos (puerto 20 para modos activos) para transferir archivos.
- **ftp (21/tcp):** Puerto de control de FTP. En este puerto se intercambian comandos y respuestas entre cliente y servidor (login, listados, inicio de transferencias, etc.).
- **fsp (21/udp):** Hace referencia a FSP (File Service Protocol), un protocolo de transferencia de archivos sobre UDP utilizado históricamente. Hoy en día es poco común.
- **ssh (22/tcp y 22/udp):** SSH (Secure Shell) usa principalmente 22/tcp para sesiones remotas seguras, túneles y transferencias (SCP/SFTP). La entrada UDP aparece por compatibilidad histórica.
- **telnet (23/tcp):** Protocolo de acceso remoto a terminales, sin cifrado. Ha sido reemplazado por SSH por razones de seguridad.
- **smtp (25/tcp):** Puerto del Simple Mail Transfer Protocol (SMTP), usado por servidores de correo para enviar y reenviar mensajes entre dominios.
- **time (37/tcp y 37/udp):** Servicio de hora simple (RFC 868), devuelve la hora en segundos desde una época fija. Reemplazado actualmente por NTP.
- **fspd:** Demonio o servicio asociado al protocolo FSP.
- **mail:** Nombre genérico que puede apuntar a servicios de mensajería o correo. Depende de la implementación.
- **timserver:** Variante histórica de servidor de tiempo, asociada al servicio `time`.

Nota: `/etc/services` contiene mapeos simbólicos nombre → puerto/protocolo. No todos los nombres aparecen en todas las distribuciones, y algunos servicios son alias o referencias históricas.

1.2 Clasificación de los protocolos de transporte

Los protocolos de transporte se pueden clasificar según dos criterios principales:

1. Orientación a conexión:

- *Con conexión*: se establece un canal lógico antes de transmitir datos.
- *Sin conexión*: los mensajes se envían de forma independiente.

2. Confiabilidad:

- *Confiables*: garantizan entrega, orden y corrección de errores.
- *No confiables*: entregan los datos sin verificación.

En el modelo TCP/IP destacan dos protocolos fundamentales:

- **TCP (Transmission Control Protocol)**: orientado a conexión, confiable, garantiza el orden y la integridad de los datos.
- **UDP (User Datagram Protocol)**: sin conexión, no confiable, utilizado en aplicaciones donde la velocidad prima sobre la confiabilidad (streaming, DNS, VoIP).

Existen variantes intermedias como **RUDP (Reliable UDP)**, que extienden a UDP para incorporar confiabilidad sin la sobrecarga de TCP.

2 El protocolo UDP

UDP (User Datagram Protocol), como hemos dicho, es un protocolo no confiable ni orientado a conexión. Está especificado en el [RFC 768](#).

Extiende el servicio de la capa de red subyacente, transformándolo en un canal de comunicación entre procesos. En otras palabras, agrega **demultiplexación a IP**.

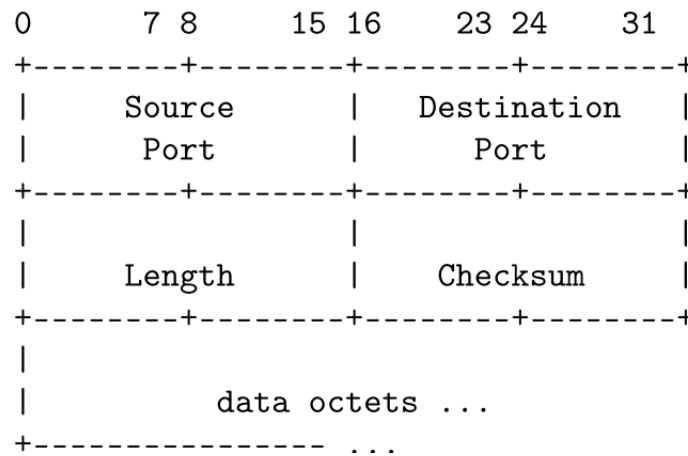
Características

- Establece una comunicación poco costosa entre procesos.
- Evita la sobrecarga y las demoras propias de la entrega ordenada y confiable de mensajes.
- Soporta múltiples procesos de nivel de aplicación en cada máquina.
- Genera tráfico egoísta: puede congestionar las colas de entrada y salida de los routers.
- No posee control de flujo: el destinatario puede verse saturado y perder información en consecuencia.
- Esencialmente, puede considerarse como **IP + puerto**.

¿Por qué UDP?

- Permite un control preciso sobre qué información se envía y cuándo se envía.
- No requiere demoras para el establecimiento de conexiones:
 - UDP no contempla ningún procedimiento preliminar formal.
- No mantiene estado de conexión:
 - No se reservan parámetros, números de secuencia, ni buffers asociados.
 - Esto facilita la manipulación de muchos clientes activos en simultáneo.
- Posee un overhead muy bajo: el encabezado ocupa sólo **8 bytes**.

2.1 Formato del datagrama UDP (según RFC 768)



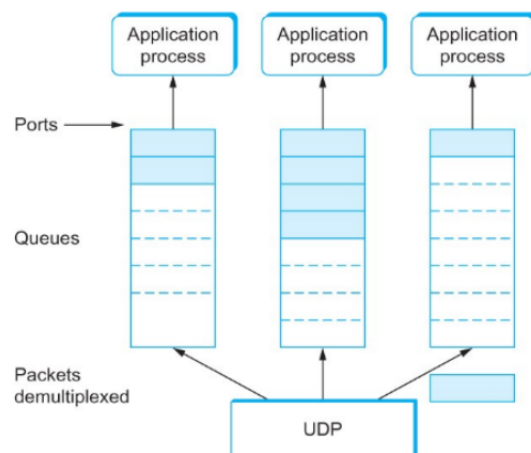
El datagrama UDP tiene una longitud mínima de 8 bytes (correspondientes al encabezado). Cada campo cumple una función específica:

- **Source Port (Puerto de origen):** identifica el puerto del proceso emisor. Puede usarse para responder al remitente; en algunos casos puede ser cero si la aplicación no necesita recibir respuestas.
- **Destination Port (Puerto de destino):** indica el puerto del proceso receptor. Permite que el sistema operativo entregue el datagrama al proceso correcto dentro de la máquina destino.
- **Length (Longitud):** especifica la longitud total del datagrama UDP en bytes, incluyendo tanto el encabezado como los datos. El tamaño mínimo posible es de 8 bytes (el tamaño del header sin datos).
- **Checksum:** valor de verificación para detectar errores en la transmisión. Se calcula sobre el encabezado, los datos y un *pseudoheader* que incluye los campos del encabezado IP:
 - Dirección IP de origen.
 - Dirección IP de destino.
 - Campo **Protocol** (que identifica UDP).

En IPv4 su uso es opcional, mientras que en IPv6 es obligatorio.

El resto del datagrama contiene los **datos de la aplicación**, también llamados *data octets*, que son entregados al proceso de destino tal como fueron enviados.

2.2 Puertos como colas de mensajes



Cuando un paquete arriba a destino, éste se dirige al puerto correspondiente y aguardará allí encolado. Si no hay espacio suficiente, el paquete se descartará sin más.

La aplicación consumirá (removerá) el primer paquete encolado al momento de recibir un mensaje. Si la cola está vacía, la aplicación quedará bloqueada hasta que llegue un nuevo paquete.

3 El Protocolo TCP

El **Transmission Control Protocol (TCP)** es el protocolo de nivel de transporte central en la arquitectura TCP/IP. Ofrece un servicio **orientado a conexión y confiable** a las aplicaciones que se ejecutan sobre él, proporcionando un canal de comunicación virtual que garantiza la entrega ordenada y libre de errores de los datos transmitidos.

TCP abstrae las características de la red subyacente, exponiendo a las capas superiores un flujo de bytes (*byte stream*) continuo, secuenciado y confiable. Fue estandarizado originalmente en el [RFC 793](#) (1981), aunque su definición ha sido actualizada en sucesivos documentos, como el [RFC 1122](#), y consolidada en un borrador reciente conocido como **RFC 793 bis**, que redefine las especificaciones de TCP en el contexto de la Internet moderna.

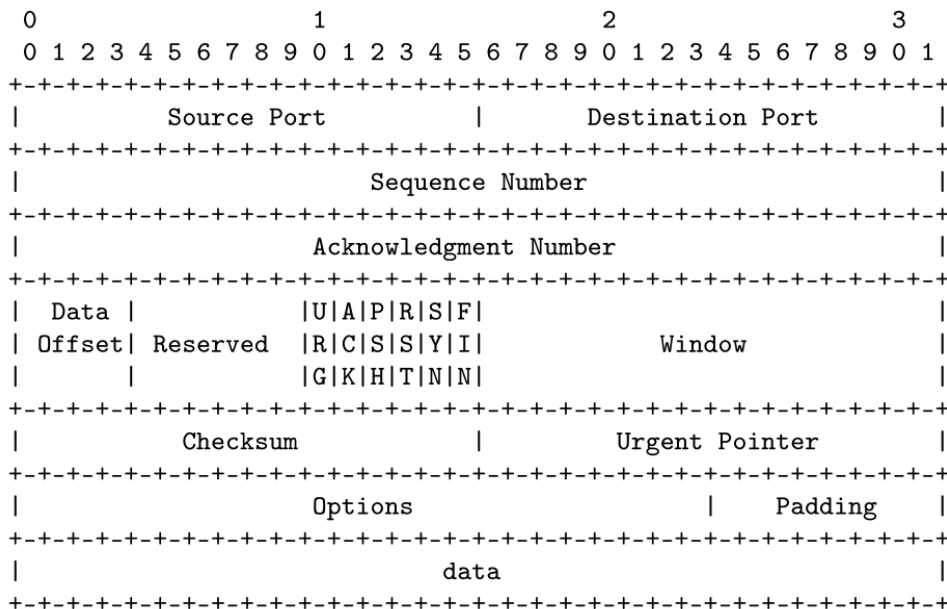
3.1 Diferencias básicas con UDP

En contraposición al protocolo UDP, TCP provee una comunicación con las siguientes características esenciales:

- **Transferencia de datos en orden:** los datos llegan al destino en el mismo orden en que fueron enviados. Esto se logra mediante el uso de *números de secuencia* incluidos en cada segmento, que permiten al receptor reordenar los datos antes de entregarlos a la aplicación.
- **Garantía de transmisión libre de errores:** TCP utiliza sumas de verificación (*checksums*) y mecanismos de retransmisión para asegurar la entrega confiable de cada byte.
- **Control de flujo:** el protocolo regula la cantidad de datos que el emisor puede enviar antes de recibir un reconocimiento, evitando la saturación del receptor. Este límite es comunicado a través del campo **Window** del encabezado TCP.
- **Retransmisión de segmentos:** cualquier byte no reconocido mediante un *ACK* dentro del tiempo esperado será retransmitido.
- **Control de congestión:** TCP adapta dinámicamente su tasa de envío para evitar la congestión de la red. (Este tema se estudia en mayor detalle por separado.)

3.2 Formato del segmento TCP

El encabezado TCP tiene la siguiente estructura (según [RFC 793](#)):



Campos principales

- **Source Port / Destination Port:** identifican los puertos de origen y destino que permiten asociar la comunicación con los procesos correspondientes en cada host.
- **Sequence Number:** indica el número de secuencia del primer byte de datos en el segmento. Cada byte transmitido en una conexión TCP es numerado secuencialmente.

- **Acknowledgment Number:** si el flag ACK está activo, este campo contiene el número de secuencia del próximo byte que el receptor espera recibir, implementando el mecanismo de *ACK acumulativos*.
- **Data Offset:** especifica la longitud del encabezado TCP en palabras de 32 bits, indicando dónde comienzan los datos.
- **Flags (Control Bits):** conjunto de bits de control que definen el propósito del segmento:
 - URG: indica que el campo **Urgent Pointer** es válido.
 - ACK: señala que el campo de reconocimiento es válido.
 - PSH: solicita al receptor entregar los datos inmediatamente a la aplicación.
 - RST: reinicia una conexión anómala.
 - SYN: sincroniza los números de secuencia durante la apertura.
 - FIN: indica que el emisor no enviará más datos.
- **Window:** informa cuántos bytes el receptor está dispuesto a aceptar, implementando el control de flujo.
- **Checksum:** cubre tanto el encabezado como los datos y un *pseudoheader* IP, permitiendo la detección de errores en la transmisión.
- **Urgent Pointer:** indica la posición del último byte de datos marcados como urgentes.
- **Options:** campo de longitud variable utilizado para negociar parámetros opcionales (por ejemplo, tamaño máximo de segmento o timestamps).
- **Data:** contiene la carga útil entregada por la capa de aplicación.

3.3 Esquema de reconocimiento

TCP utiliza un mecanismo de **reconocimiento acumulativo**: un número de ACK igual a n indica que el receptor ha recibido correctamente todos los bytes numerados desde 0 hasta $n - 1$, y que espera el byte número n como siguiente.

Si un ACK no llega dentro del tiempo estimado, los datos pendientes se retransmiten. El tiempo de retransmisión (*RTO*) se calcula a partir del *Round-Trip Time* (*RTT*) estimado para la conexión. TCP también puede utilizar **reconocimientos selectivos** (SACK) mediante opciones específicas del encabezado.

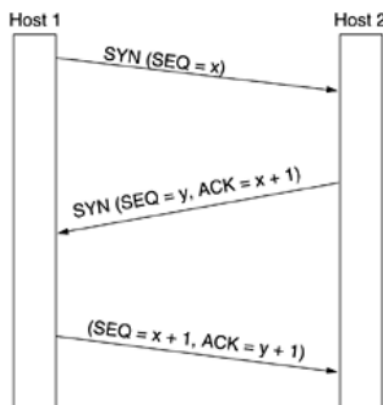
3.4 Modo de operación

TCP opera en tres fases bien definidas:

1. **Establecimiento de conexión:** ambas partes sincronizan números de secuencia iniciales y crean el estado necesario para la sesión.
2. **Transferencia de datos:** los hosts intercambian segmentos con control de flujo y retransmisión.
3. **Cierre de conexión:** los extremos liberan los recursos y finalizan la sesión.

Durante estas fases, la conexión transita diferentes estados definidos por una máquina de estados finita, en la que distintos eventos (envío, recepción, timeout) provocan transiciones específicas.

3.5 Establecimiento de conexión: Three-Way Handshake

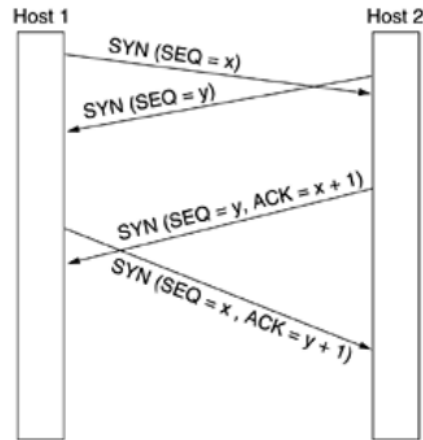


El **three-way handshake** es el mecanismo mediante el cual TCP establece una conexión entre un cliente y un servidor, garantizando la sincronización de los números de secuencia iniciales (ISN) de ambos extremos.

- El servidor realiza un *passive open*, asociándose a un puerto local y quedando a la espera de solicitudes (LISTEN).
- El cliente inicia un *active open*, enviando un segmento con el flag SYN activado.
- El servidor responde con un SYN+ACK, indicando su propio número de secuencia y reconociendo el del cliente.
- Finalmente, el cliente envía un ACK final, completando el establecimiento de la conexión.

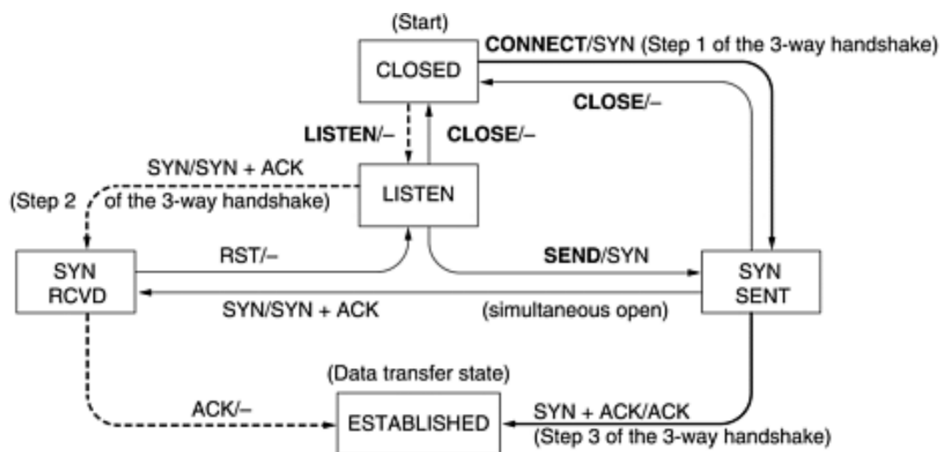
El flag SYN es secuenciable: ocupa un número de secuencia propio. Si el puerto solicitado no está escuchando, el sistema operativo responde con un paquete con el flag RST activado.

3.6 Apertura simultánea



Aunque infrecuente, TCP contempla la posibilidad de una **apertura simultánea**, en la cual ambos hosts envían un SYN de forma concurrente. En tal caso, cada extremo responde al SYN de la contraparte con un SYN+ACK; al recibir el reconocimiento, ambos consideran la conexión establecida.

3.7 Diagrama de estados: apertura



El establecimiento de una conexión TCP está gobernado por una **máquina de estados finita (FSM, Finite State Machine)** que define el comportamiento de cada extremo (cliente o servidor) ante distintos eventos, como el envío o recepción de paquetes con ciertos flags, o el vencimiento de temporizadores.

Durante la **fase de apertura**, el objetivo es transicionar desde un estado pasivo o inactivo (CLOSED) hasta el estado operativo (ESTABLISHED), garantizando que ambos extremos hayan sincronizado correctamente sus números de secuencia.

Los estados y transiciones principales según Tanenbaum:

- **CLOSED:** estado inicial, sin conexión activa. Desde aquí, el servidor puede ejecutar un **passive open** (llamada LISTEN) o el cliente puede iniciar un **active open** (enviando un segmento SYN).
- **LISTEN:** el servidor se encuentra esperando solicitudes entrantes en un puerto determinado.
- **SYN_SENT:** el cliente ha enviado un SYN y espera recibir un SYN+ACK del servidor. Si lo recibe, responde con un ACK y pasa a ESTABLISHED.
- **SYN_RECEIVED:** el servidor ha recibido un SYN, respondió con SYN+ACK, y aguarda el ACK final del cliente.
- **ESTABLISHED:** la conexión está activa; ambas partes pueden enviar y recibir datos. Este es el estado operativo normal del protocolo.

En resumen, el proceso de apertura normal (three-way handshake) implica tres segmentos y tres transiciones: SYN → SYN+ACK → ACK, estableciendo la sesión full-duplex con sincronización de secuencias en ambos sentidos.

3.8 Cierre de conexión: Four-Way Handshake

El cierre de una conexión TCP puede ser **asimétrico** o **simétrico**. Para realizarlo correctamente, se utiliza un procedimiento conocido como **four-way handshake**.

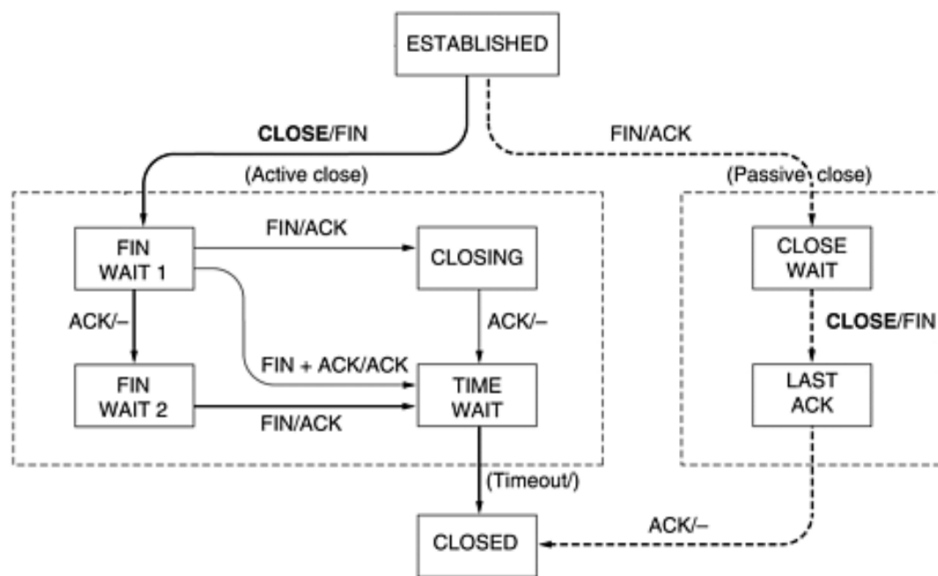
- Un extremo envía un segmento con el flag FIN activado, indicando que no transmitirá más datos.
- El receptor responde con un ACK, reconociendo el cierre parcial del flujo.
- Posteriormente, el receptor envía su propio FIN, el cual debe ser reconocido por el otro extremo.

En los cierres simétricos, los FIN y ACK suelen combinarse en mensajes FIN+ACK. El flag FIN, al igual que SYN, consume un número de secuencia.

3.9 Cierre simultáneo

TCP también contempla el caso de un **cierre simultáneo**, donde ambos extremos envían sus segmentos FIN de manera independiente. En esta situación, la transición de estados difiere ligeramente del caso asimétrico, pero el resultado final es el mismo: la conexión se cierra correctamente tras el intercambio de reconocimientos.

3.10 Diagrama de estados: cierre



El **cierre de la conexión TCP** también se modela mediante una máquina de estados, cuyo objetivo es garantizar que ambos extremos reconozcan la finalización del flujo de datos, evitando la pérdida o duplicación de información.

A diferencia del establecimiento, el cierre puede ser **asimétrico** (unidireccional) o **simétrico** (mutuo), y utiliza el mecanismo de *four-way handshake* basado en los flags FIN y ACK.

Los estados más relevantes son los siguientes:

- **ESTABLISHED:** la conexión está abierta y se intercambian datos normalmente.
- **FIN_WAIT_1:** el extremo que inicia el cierre ha enviado un segmento FIN y espera su reconocimiento (ACK).
- **FIN_WAIT_2:** se recibió el ACK del FIN enviado, pero aún se esperan los datos y el FIN del otro extremo.
- **CLOSE_WAIT:** el extremo receptor del FIN lo ha reconocido con un ACK, pero su aplicación aún no ha cerrado el flujo de salida.
- **LAST_ACK:** el extremo en CLOSE_WAIT ha enviado su propio FIN y espera el ACK final.
- **TIME_WAIT:** el iniciador del cierre ha recibido el FIN del otro extremo, respondió con ACK, y permanece un tiempo ($2 \times \text{RTT}$) para asegurar que todos los segmentos pendientes hayan expirado.
- **CLOSED:** estado final, sin recursos asociados a la conexión.

El estado **TIME_WAIT** es fundamental: garantiza que los últimos ACK o retransmisiones no sean confundidos con nuevas conexiones reutilizando el mismo par <IP, puerto>.

En caso de **cierre simultáneo**, ambos extremos envían su FIN antes de recibir el del otro, ingresando temporalmente en **CLOSING** antes de pasar a **TIME_WAIT** y finalmente a **CLOSED**.

4 Ejercicio 1: análisis de una secuencia de segmentos TCP

Dada la siguiente secuencia de segmentos TCP, responder las preguntas que siguen:

#	ORIG	DEST	FLAGS	#SEQ	#ACK	LENGTH
1	1.2.3.4:5678	20.232.1.1:80	S	0	—	0
2	1.1.1.1:2222	3.3.3.3:4444	S	42	—	0
3	20.232.1.1:80	1.2.3.4:5678	SA	10000	1	0
4	1.2.3.4:5678	20.232.1.1:80	A	1	10001	0
5	3.3.3.3:4444	1.1.1.1:2222	SA	54321	43	0
6	1.2.3.4:5678	20.232.1.1:80	A	1	10001	50
7	1.1.1.1:2222	3.3.3.3:4444	A	43	54322	0
8	20.232.1.1:80	1.2.3.4:5678	A	10001	51	0
9	20.232.1.1:80	1.2.3.4:5678	A	10001	51	200
10	1.1.1.1:2222	3.3.3.3:4444	SAU	43	64334	0
11	20.232.1.1:80	1.2.3.4:5678	F	10201	—	0
12	1.2.3.4:5678	20.232.1.1:80	FA	51	10202	0
13	3.3.3.3:4444	1.1.1.1:2222	RA	54321	43	0
14	20.232.1.1:80	1.2.3.4:5678	A	10202	52	0

1. Asociar cada segmento con cada una de las conexiones indicando el criterio usado para determinar a qué conexión pertenece un segmento. ¿Cuántas son?
2. Detectar el cierre anómalo de una de las conexiones e indicar una posible causa.
3. Para la otra conexión, detallar los cambios de estado en cada extremo a lo largo de toda la comunicación.

Resolución

Para responder cuántas conexiones TCP hay, tal vez sea conveniente mirar primero la segunda pregunta del ejercicio. En general, al momento de determinar la conexión asociada a un paquete dado, hay que mirar cuatro componentes: dirección IP origen, puerto origen, dirección IP destino y puerto destino. Con esta tupla de valores podemos caracterizar el flujo al que pertenece el paquete. En este caso, como los orígenes son distintos, basta con mirar sólo la IP origen. Dicho esto, podemos concluir que en la captura aparecen dos conexiones: (1.2.3.4:5678, 20.232.1.1:80) y (1.1.1.1:2222, 3.3.3.3:4444).

Respecto al cierre anómalo, puede verse que éste ocurre en el paquete 13, cuando el proceso en 3.3.3.3:4444 envía un RST. Esto se debe al paquete con flags SYN/ACK/URG recibido en la línea 10: recibir un SYN en una conexión sincronizada tiene como efecto el envío de un RST, conforme a la especificación de TCP (para más detalles, referirse a la página 71 del RFC 793). La consecuencia de este envío es que el receptor procederá a liberar los recursos de la conexión de inmediato, sin ejercer el algoritmo de cierre tradicional. A su vez, el host emisor del RST hace esto mismo en reacción al paquete mal formado que recibió.

Veamos finalmente cómo responder la última pregunta. Para esto, separemos primero los paquetes de la conexión:

#	ORIG	DEST	FLAGS	#SEQ	#ACK	LENGTH
1	1.2.3.4:5678	20.232.1.1:80	S	0	—	0
3	20.232.1.1:80	1.2.3.4:5678	SA	10000	1	0
4	1.2.3.4:5678	20.232.1.1:80	A	1	10001	0
6	1.2.3.4:5678	20.232.1.1:80	A	1	10001	50
8	20.232.1.1:80	1.2.3.4:5678	A	10001	51	0
9	20.232.1.1:80	1.2.3.4:5678	A	10001	51	200
11	20.232.1.1:80	1.2.3.4:5678	F	10201	—	0
12	1.2.3.4:5678	20.232.1.1:80	FA	51	10202	0
14	20.232.1.1:80	1.2.3.4:5678	A	10202	52	0

Ahora, analicemos paquete por paquete qué está sucediendo en cada extremo:

- Del paquete 1 se desprende que 1.2.3.4:5678 está haciendo una apertura activa. Por ende, este paquete hace que su TCP se mueva de **CLOSED** a **SYN-SENT**.
- El paquete 3 es una respuesta positiva del paquete 1 por parte del proceso 20.232.1.1:80. Para que esto ocurra, su TCP debe haber estado en **LISTEN** antes de poder contestar. Al recibir el **SYN**, la reacción consiste en trasladarse a **SYN-RCVD** y emitir un paquete **SYN+ACK**.
- El primer host recibe luego este segmento y esto hace que se mueva a **ESTABLISHED**, enviando como respuesta el **ACK** final del three-way handshake (paquete 4).
- Una vez procesado esto, el TCP de 20.232.1.1:80 se moverá también a **ESTABLISHED**.
- Los paquetes 6, 8 y 9 corresponden a envío y reconocimiento de datos, por lo que no motivan transiciones entre estados.
- En la porción final de la captura, se inicia el proceso de cierre de conexión. El segmento 11 nos muestra que 20.232.1.1:80 está haciendo un cierre activo, revirtiendo así su rol “pasivo” al momento de establecer la conexión. El efecto de este envío es transicionar al estado **FIN-WAIT-1**.
- En la siguiente línea, su interlocutor recibe este mensaje y reacciona respondiendo un paquete **FIN+ACK** que no sólo reconoce el primer **FIN** (puesto que su **#ACK** es 10202), sino que además inicia el cierre de su stream de escritura. En consecuencia, su TCP colapsa dos estados, pasando de **ESTABLISHED** a **LAST-ACK**.
- El segmento 14 reconoce correctamente el **FIN** de 1.2.3.4:5678. Al recibirlo, el TCP de dicho proceso transicionará a **CLOSED**, mientras que el de 20.232.1.1:80 se moverá a **TIME-WAIT** como respuesta al segmento 12. Asumiendo que el último **ACK** no se extravía en la red, allí permanecerá por 120 segundos (i.e., 2 MSLs) antes de pasar también a **CLOSED**.

5 Ejercicio 2: Captura de paquetes TCP con cierre de conexión

La siguiente captura de paquetes TCP corresponde a un programa corriendo en un host A comunicándose con el servicio corriendo en el puerto 5900 del host B:

No.	Source	Dest	Info
1	A	B	26574 > 5900 [SYN] Seq=0 Ack=0 Len=0
2	B	A	5900 > 25674 [SYN,ACK] Seq=0 Ack=1 Len=0
3	A	B	26574 > 5900 [ACK] Seq=1 Ack=1 Len=0
4	A	B	26574 > 5900 [PSH,ACK] Seq=1 Ack=1 Len=1024
5	B	A	5900 > 25674 [ACK] Seq=1 Ack=1025 Len=200
6	A	B	26574 > 5900 [ACK] Seq=1025 Ack=201 Len=0

Extender la captura proponiendo una serie de paquetes que hagan que el socket del host A atravesase los siguientes estados:

ESTABLISHED → FIN-WAIT-1 → FIN-WAIT-2 → TIME-WAIT → CLOSED

Antes de mostrar los segmentos que satisfagan lo pedido, observemos lo siguiente:

- Para que el host A pase de **ESTABLISHED** a **FIN-WAIT-1**, debe ocurrir que tome una postura activa y decida cerrar la conexión primero.
- Si el siguiente estado debe ser **FIN-WAIT-2**, necesitamos que el host B responda simplemente con **ACK** al **FIN** original.

- Luego, al cabo de una cantidad arbitraria de paquetes de datos por parte de B, debe darse la transición a **TIME-WAIT** de A. Esto lo lograremos haciendo que B envíe su **FIN**.
- Naturalmente, A debe reconocer correctamente este **FIN**, lo cual motivará su paso a **TIME-WAIT**.
- La transición final a **CLOSED** se va a dar implícitamente; no debemos agregar ningún otro paquete para alcanzar este objetivo.

Con esto en mente, una posible solución podría ser la que se detalla a continuación. Notar que la respuesta no es única dado que B puede enviar tantos datos como se quiera. Prestar también especial atención a los números de secuencia y de reconocimiento utilizados, recordando que el flag de **FIN** es secuenciable.

No.	Source	Dest	Info
7	A	B	26574 >5900 [FIN,ACK] Seq=1025 Ack=201 Len=0
8	B	A	5900 >25674 [ACK] Seq=201 Ack=1026 Len=0
9	B	A	5900 >26574 [FIN,ACK] Seq=201 Ack=1026 Len=0
10	A	B	26574 >5900 [ACK] Seq=1026 Ack=202 Len=0

En este escenario, como dijimos, el host A es el que inicia activamente el cierre de conexión.

Si invertimos este comportamiento, haciendo que sea B el que lo inicie, obtendremos otro escenario de cierre válido. También podría ocurrir que ambos hosts decidan cerrar la conexión simultáneamente, lo cual daría origen a otro escenario de cierre distinto. Este último caso tiene la particularidad de que ambos hosts transitarán por los mismos estados: de **ESTABLISHED** saltarán a **FIN-WAIT-1** al enviar el **FIN**, para luego trasladarse a **CLOSING** al recibir el **FIN** de la contraparte. La respuesta de sendos **FINs** hará que, finalmente, los interlocutores pasen a **TIME-WAIT** y, al cabo de 2 MSLs, a **CLOSED**.

6 Port Scanning

La identificación de los puertos disponibles en un host se conoce como **port scanning**. Este procedimiento consiste en barrer una secuencia de puertos enviando paquetes y analizando las posibles respuestas, y puede materializarse de formas variadas según la heurística que se desee utilizar. Hoy mencionaremos los dos tipos de escaneo más importantes: **SYN scan** y **connect scan**. Ambos se apoyan fuertemente en la compatibilidad con la especificación de TCP, detallada en el [RFC 793](#).

6.1 Clasificación de puertos

Usualmente, un escaneo clasificará a un puerto de la siguiente manera:

- **open (abierto)**: si pudo inferir de su heurística que hay un servicio escuchando en dicho puerto.
- **closed (cerrado)**: si determinó que no hay un proceso escuchando allí.
- **filtered (filtrado)**: si hay chances de que alguna entidad intermedia (por ejemplo, un firewall) descarte el tráfico correspondiente.

Puede ocurrir que la heurística del scanner no pueda discernir completamente entre estas categorías.

6.2 SYN scan (o half-open scan)

El SYN scan es el método más popular. Consiste en enviar un paquete TCP con el flag **SYN** y esperar la respuesta:

- **SYN/ACK**: indica que el puerto está **abierto**.
- **RST**: indica que el puerto está **cerrado**.
- **No hay respuesta** o un **ICMP destination unreachable**: se asume que el puerto está **filtrado**.

En este método no se completa la conexión, lo que lo hace más discreto.

Ventajas y desventajas del SYN scan

- **Ventajas**: rápido, poco invasivo y discreto al no completar las conexiones.
- **Desventajas**: requiere enviar y recibir paquetes ad-hoc, lo cual puede requerir permisos elevados en el sistema.

6.3 Connect scan (o TCP scan)

El connect scan utiliza la API del sistema operativo para establecer una conexión a través de la primitiva `connect` de sockets. Este método inicia la conexión de manera convencional, siguiendo el algoritmo del **three-way handshake** detallado en el [RFC 793](#).

Ventajas y desventajas del Connect scan

- **Ventajas:** no requiere permisos elevados para ejecutarse.
- **Desventajas:** más lento y con mayor volumen de tráfico, al completar conexiones; menos discreto, ya que los hosts pueden registrar estas conexiones en sus logs.

7 Herramientas de análisis TCP/IP

En esta sección se presentan herramientas fundamentales para el análisis de conexiones TCP/IP, escaneo de puertos y diagnóstico de servicios.

7.1 Nmap

Nmap es una herramienta de descubrimiento de redes muy completa, lanzada en 1997. Permite:

- Discovery de hosts.
- Escaneo de puertos (SYN scan, Connect scan y otras técnicas avanzadas).
- Detección de servicios y sistemas operativos.
- Ejecución de scripts custom.

Su nombre proviene de *Network Mapper*. La siguiente corrida demuestra un escaneo de puertos y detección de servicios:

```
1 lucio@knuth:~$ sudo nmap -sS -sV 192.168.1.100
2 Starting Nmap 5.21 ( http://nmap.org ) at 2012-10-30 01:56 ART
3 Nmap scan report for 192.168.1.100
4 Host is up (0.013s latency).
5 Not shown: 998 closed ports
6 PORT      STATE SERVICE VERSION
7 22/tcp    open  ssh      OpenSSH 5.3p1 Debian 3ubuntu6 (protocol 2.0)
8 80/tcp    open  http?    1 service unrecognized despite returning data
9 MAC Address: 00:24:8C:96:57:8B (Asustek Computer)
10 Service Info: OS: Linux
11 Service detection performed. Please report any incorrect results at http://nmap.org/submit/.
12 Nmap done: 1 IP address (1 host up) scanned in 134.77 seconds
```

- `lucio@knuth: $ sudo nmap -sS -sV 192.168.1.100`: ejecución del comando `nmap` con privilegios de administrador.
 - `-sS`: realiza un escaneo *SYN* (half-open), más sigiloso que una conexión completa.
 - `-sV`: habilita la detección de versiones de los servicios que responden en los puertos abiertos.
 - `192.168.1.100`: dirección IP del host objetivo.
- `Starting Nmap 5.21 ( http://nmap.org ) at 2012-10-30 01:56 ART`: indica la versión de Nmap, la URL oficial del proyecto y la fecha/hora de inicio del escaneo.
- `Nmap scan report for 192.168.1.100`: encabezado del informe; muestra el objetivo escaneado.
- `Host is up (0.013s latency).`: confirma que el host respondió y estima la latencia (13 ms en este caso).
- `Not shown: 998 closed ports`: resume que, de todos los puertos probados, 998 están cerrados y no devuelven respuesta.
- `PORT STATE SERVICE VERSION`: encabezado de la tabla que resume el estado de los puertos y los servicios detectados.
- `22/tcp open ssh OpenSSH 5.3p1 Debian 3ubuntu6 (protocol 2.0)`:
 - Puerto `22/tcp` abierto, correspondiente al servicio SSH.
 - Se detectó la versión `OpenSSH 5.3p1` de la distribución Debian (paquete `3ubuntu6`).

- Indica que el servicio usa el protocolo SSH versión 2.0.
- 80/tcp open http? 1 service unrecognized despite returning data.:
 - Puerto 80/tcp abierto, identificado tentativamente como HTTP.
 - Nmap recibió datos, pero no logró reconocer con certeza el tipo de servicio (por eso el “?”).
- MAC Address: 00:24:8C:96:57:8B (Asustek Computer): dirección MAC del host detectado y su fabricante (Asustek Computer).
- Service Info: OS: Linux: información adicional inferida: el sistema operativo del host parece ser Linux.
- Service detection performed. Please report any incorrect results at <http://nmap.org/submit/>.: aviso estándar de Nmap invitando a reportar errores en la detección de servicios.
- Nmap done: 1 IP address (1 host up) scanned in 134.77 seconds: resumen final; Nmap escaneó un único host activo y el proceso demoró 134.77 segundos.

7.2 Telnet

Telnet es un protocolo de aplicación sobre TCP que permite conectarse a un host remoto y usar una interfaz de línea de comandos. Hoy se utiliza principalmente para pruebas de conectividad, no para acceso remoto por ser inseguro. Ejemplo:

```
1 lucio@knuth:~$ telnet proxymail1.sion.com 25
2 Trying 200.81.186.15...
3 Connected to proxymail1.sion.com.
4 Escape character is '^]'.
5 220 mx2.sion.com ESMTP (SION)
6 QUIT
7 221 2.0.0 Bye
8 Connection closed by foreign host.
```

- lucio@knuth: \$ telnet proxymail1.sion.com 25: comando que invoca **telnet** para conectarse al puerto 25 (SMTP) del host **proxymail1.sion.com**. Telnet abre una conexión TCP hacia el servicio destino y permite interacción manual con el protocolo de aplicación.
- Trying 200.81.186.15...: mensaje del cliente indicando la IP resuelta para el nombre de host y que está intentando establecer la conexión TCP.
- Connected to proxymail1.sion.com.: confirmación de que la conexión TCP se estableció con éxito (completado el three-way handshake).
- Escape character is '^]': telnet imprime cuál es la secuencia de escape para volver al prompt del cliente (útil para cerrar o suspender la sesión interactiva).
- 220 mx2.sion.com ESMTP (SION): banner inicial enviado por el servidor SMTP al establecer la conexión. El código 220 indica que el servicio está listo; la línea normalmente incluye el identificador del servidor y el tipo (ESMTP).
- QUIT: comando enviado por el usuario (cliente telnet) para indicar al servidor SMTP que desea terminar la sesión de forma ordenada.
- 221 2.0.0 Bye: respuesta del servidor SMTP al comando QUIT. El código 221 confirma el cierre ordenado del servicio y el mensaje adicional ofrece una razón/estado.
- Connection closed by foreign host.: notificación del cliente telnet indicando que el host remoto cerró la conexión TCP; la socket local pasa a estado cerrado y la sesión interactiva finaliza.

7.3 Netstat

Netstat permite visualizar todas las conexiones TCP/UDP y su estado, así como los procesos asociados a cada socket. Ejemplo de salida:

```
1 lucio@knuth:~$ netstat -an | more
2 Proto Local Address      Foreign Address    State
3 tcp    127.0.0.1:631    0.0.0.0:*          LISTEN
4 tcp    127.0.0.1:5432   0.0.0.0:*          LISTEN
5 tcp    0.0.0.0:902      0.0.0.0:*          LISTEN
6 tcp    0.0.0.0:80       0.0.0.0:*          LISTEN
7 tcp    192.168.1.101:46670 65.55.71.176:1863 ESTABLISHED
8 tcp    192.168.1.101:37853 74.125.130.125:5222 ESTABLISHED
9 tcp    192.168.1.101:57385 192.168.1.100:22   ESTABLISHED
```

- **lucio@knuth: \$ netstat -an | more:** comando que ejecuta **netstat** mostrando todas las conexiones TCP/UDP y los sockets en estado numérico (-n) y paginando la salida con **more**.
- **Proto:** indica el protocolo asociado a cada socket (TCP o UDP).
- **Local Address:** dirección IP y puerto local del socket en la máquina donde se ejecuta **netstat**.
- **Foreign Address:** dirección IP y puerto remoto al que está conectado el socket, si aplica; un asterisco (*) indica “cualquier host” o puerto no asignado.
- **tcp 127.0.0.1:631 0.0.0.0:*:** socket TCP local en el puerto 631 (CUPS), escuchando en localhost; el estado listado luego es **LISTEN**.
- **tcp 127.0.0.1:5432 0.0.0.0:*:** socket TCP en puerto 5432 (PostgreSQL), escuchando en localhost; estado **LISTEN**.
- **tcp 0.0.0.0:902 0.0.0.0:*:** socket TCP escuchando en todas las interfaces disponibles en el puerto 902 (posible servicio VMWare o similar); estado **LISTEN**.
- **tcp 0.0.0.0:80 0.0.0.0:*:** socket TCP escuchando en todas las interfaces en el puerto 80 (HTTP); estado **LISTEN**.
- **tcp 192.168.1.101:46670 65.55.71.176:1863:** conexión TCP activa desde un puerto efímero local hacia el host remoto en puerto 1863 (posiblemente MSN Messenger o similar); estado **ESTABLISHED**.
- **tcp 192.168.1.101:37853 74.125.130.125:5222:** conexión TCP establecida hacia el host remoto en puerto 5222 (XMPP/GTalk); estado **ESTABLISHED**.
- **tcp 192.168.1.101:57385 192.168.1.100:22:** conexión TCP hacia el puerto 22 de un host remoto (SSH); estado **ESTABLISHED**.
- **State LISTEN / ESTABLISHED:** listado final de estados para cada socket, mostrando cuáles están esperando conexiones (**LISTEN**) y cuáles tienen conexión activa (**ESTABLISHED**).

7.4 Netcat

netcat es la navaja suiza de TCP/IP. Permite:

- Escaneo de puertos.
- Transferencia de archivos.
- Conexión a servidores (similares a Telnet).
- Escuchar en un puerto dado.

```

1 lucio@knuth:~$ nc -vz 192.168.1.100 77-83
2 nc: connect to 192.168.1.100 port 77 (tcp) failed
3 nc: connect to 192.168.1.100 port 78 (tcp) failed
4 nc: connect to 192.168.1.100 port 79 (tcp) failed
5 Connection to 192.168.1.100 80 port [tcp/www] succeeded!
6 nc: connect to 192.168.1.100 port 81 (tcp) failed
7 nc: connect to 192.168.1.100 port 82 (tcp) failed
8 nc: connect to 192.168.1.100 port 83 (tcp) failed

```

- **lucio@knuth: \$ nc -vz 192.168.1.100 77-83:** ejecución de **netcat** en modo verboso (-v) y verificación de conexión (-z) sobre el rango de puertos 77–83 hacia la IP objetivo. **nc** intentará conectar a cada puerto del rango.
- **nc: connect to 192.168.1.100 port 77 (tcp) failed:** intento de conexión al puerto 77 fallido — puede indicar que no hay servicio escuchando o que el paquete fue filtrado/dropped.
- **nc: connect to 192.168.1.100 port 78 (tcp) failed:** intento de conexión al puerto 78 fallido — misma interpretación que el puerto 77.
- **nc: connect to 192.168.1.100 port 79 (tcp) failed:** intento de conexión al puerto 79 fallido — no responde un servicio o hay filtrado.
- **Connection to 192.168.1.100 80 port [tcp/www] succeeded!:** conexión exitosa al puerto 80 (servicio HTTP). Netcat pudo completar la conexión TCP, lo que indica que hay un servicio escuchando en ese puerto.

- `nc: connect to 192.168.1.100 port 81 (tcp) failed:` intento al puerto 81 fallido — sin servicio o filtrado en ese puerto.
- `nc: connect to 192.168.1.100 port 82 (tcp) failed:` intento al puerto 82 fallido — misma interpretación.
- `nc: connect to 192.168.1.100 port 83 (tcp) failed:` intento al puerto 83 fallido — sin servicio escuchando o paquete descartado por algún filtro.

Netcat recorrió el rango 77–83 y detectó que solo el puerto 80 responde (abierto). Los demás puertos devolvieron fallo de conexión, lo que típicamente se categoriza como `closed` o `filtered` según el comportamiento de la red/firewall.

```
1 lucio@knuth:~$ nc -l localhost 12345 > file &
2 [1] 5557
3 lucio@knuth:~$ echo "eh amigo" > eh_amigo
4 lucio@knuth:~$ nc localhost 12345 < eh_amigo
5 [1]+ Done
6 nc -l localhost 12345 > file
7 lucio@knuth:~$ cat file
8 eh amigo
```

- `lucio@knuth: $ nc -l localhost 12345 > file &`: se lanza `netcat` en modo `listen` (`-l`) en la interfaz `localhost` y puerto `12345`; la salida estándar del proceso (todo lo que reciba) se redirige al archivo `file`. El ampersand final ejecuta el listener en segundo plano (background).
- `[1] 5557:` respuesta del shell indicando que el job ha sido creado: `[1]` es el identificador del job y `5557` es el PID del proceso en background.
- `lucio@knuth: $ echo "eh amigo" > eh_amigo`: creación de un fichero local llamado `eh_amigo` cuyo contenido es la cadena `eh amigo`; la redirección `>` escribe ese texto en el fichero.
- `lucio@knuth: $ nc localhost 12345 < eh_amigo`: uso de `netcat` en modo cliente que se conecta a `localhost:12345` y toma su entrada estándar desde el fichero `eh_amigo` (`<`). Al establecer la conexión, el contenido del fichero se envía al listener que corre en background.
- `[1]+ Done:` notificación del shell que el job `[1]` (el listener) terminó; esto ocurre porque la transferencia finalizó y el proceso `nc -l` cerró al recibir EOF desde el cliente.
- `nc -l localhost 12345 > file:` eco de la línea de comando del job que terminó (el shell muestra el comando asociado al job para claridad).
- `lucio@knuth: $ cat file`: comando para mostrar el contenido del archivo `file`, donde el listener había volcado la entrada recibida.
- `eh amigo`: salida resultante: el contenido recibido por el listener (`eh amigo`) se guardó correctamente en `file`, y `cat` muestra que la transferencia fue exitosa y sin alteraciones.

ss

ss (socket statistics) permite investigar el estado interno de los sockets TCP/UDP, incluyendo RTT, RTO, congestión y retransmisiones. Más información en [Cyberciti](#). Ejemplo:

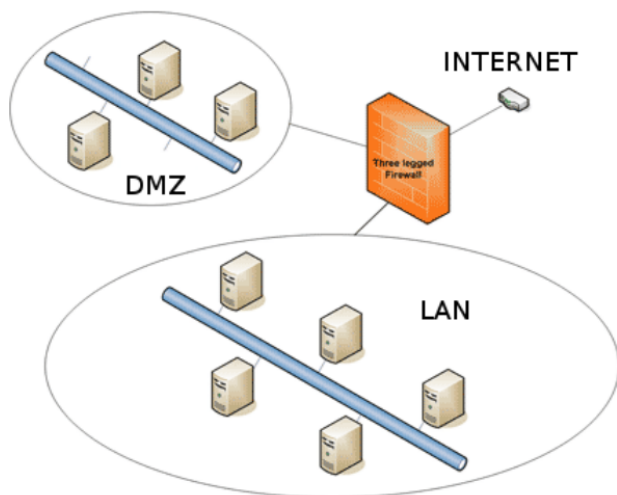
```
1 Recv-Q Send-Q Local Address:Port Peer Address:Port
2 0 0 127.0.0.1:33088 127.0.0.1:5555
3 cubic wscale:7,7 rto:220 rtt:21.5/10 mss:65483
4 cwnd:27 ssthresh:26 send 657.9Mbps rcv_space:43690
```

- **Recv-Q Send-Q**: cabecera de columnas que indica las colas de recepción (**Recv-Q**) y de envío (**Send-Q**) del socket. Estos valores muestran cuántos bytes están pendientes por ser leídos por la aplicación (Recv-Q) y cuántos bytes están pendientes de ser enviados/confirmados (Send-Q).
- **Local Address:Port**: encabezado que indica la dirección local y el puerto usados por el socket en la máquina que ejecuta `ss`.
- **Peer Address:Port**: encabezado que indica la dirección y puerto remotos (peer) con los que está establecida la conexión.

- 0: valor bajo **Recv-Q** (0) que indica que no hay datos pendientes en la cola de recepción: la aplicación ya ha leído todo lo recibido.
- 0: valor bajo **Send-Q** (0) que indica que no hay datos pendientes por enviar o por confirmar (ACK) en la cola de envío.
- 127.0.0.1:33088: dirección local y puerto del socket (localhost:33088). Es el extremo local de la conexión.
- 127.0.0.1:5555: dirección y puerto del peer remoto (en este caso también localhost:5555). Indica el extremo con el que se comunica el socket local.
- **cubic wscale:7,7 rto:220 rtt:21.5/10 mss:65483**: línea con parámetros TCP internos:
 - **cubic**: algoritmo de control de congestión en uso (CUBIC).
 - **wscale:7,7**: factores de escalado de ventana (window scaling) negociados para ambos extremos (7 para local y 7 para peer), que permiten ventanas de recepción mayores a 65 535 bytes.
 - **rto:220**: Retransmission Timeout estimado, en milisegundos (el tiempo que espera antes de retransmitir).
 - **rtt:21.5/10**: estimación del Round-Trip Time (RTT) y su varianza/medida auxiliar (aquí 21.5 ms y un segundo valor 10 — según la implementación).
 - **mss:65483**: Maximum Segment Size negociado (bytes) — tamaño máximo de datos por segmento TCP sin contar cabeceras.
- **cwnd:27 ssthresh:26 send 657.9Mbps rcv_space:43690**: métricas de control de flujo y congestión:
 - **cwnd:27**: tamaño actual de la **congestion window** (en segmentos TCP o unidades internas), que limita cuántos datos pueden estar en vuelo sin confirmar.
 - **ssthresh:26**: **slow-start threshold**, umbral que determina si TCP está en slow-start (por debajo) o en congestión avoidance (por encima).
 - **send 657.9Mbps**: estimación del throughput o tasa efectiva de envío calculada por la implementación TCP (puede ser un promedio instantáneo).
 - **rcv_space:43690**: tamaño del buffer de recepción disponible (espacio que el peer anuncia o que queda libre para recibir datos).

8 Firewall

Un **firewall** es un conjunto de sistemas encargados de mediar todas las comunicaciones entre dos redes, implementando una política de seguridad que controla el acceso mediante el *filtrado de paquetes*. Su función principal es inspeccionar cada paquete de forma independiente y decidir si permitir o denegar su paso según un conjunto de reglas predefinidas. Esta capacidad lo convierte en un componente esencial para proteger redes internas de accesos no autorizados o tráfico malicioso.



8.1 Tipos de Firewalls

- **Stateless:** Este tipo de firewall opera de manera simple y directa, evaluando cada mensaje entrante o saliente individualmente. Se basa en reglas que usualmente incluyen campos como:
 - Dirección de origen (`dir_origen`)
 - Puerto de origen (`puerto_origen`)
 - Dirección de destino (`dir_dst`)
 - Puerto de destino (`puerto_dst`)
 - Protocolo (`protocolo`)

Al no mantener estado de las conexiones, cada paquete se analiza como si fuera independiente de los anteriores.

- **Stateful:** A diferencia del *stateless*, este firewall mantiene un seguimiento del estado de las conexiones entrantes y salientes. Esto le permite decidir si permitir o denegar un paquete considerando la sesión a la que pertenece, proporcionando así un nivel superior de control y seguridad. Por ejemplo, un paquete TCP solo será aceptado si forma parte de una conexión previamente establecida y autorizada.
- **Gateways de aplicación (Application Gateways o Proxy):** Este tipo de firewall actúa como un *proxy inteligente* que entiende ciertos protocolos de aplicación. Permite aplicar filtros de capa de aplicación, inspeccionando el contenido de las comunicaciones más allá de las cabeceras de los paquetes. Esto es útil, por ejemplo, para filtrar comandos de FTP, correos electrónicos (SMTP), o tráfico web (HTTP/HTTPS), garantizando que solo se cumplan políticas específicas sobre el contenido, no solo sobre las direcciones o puertos.

En conjunto, estos tipos de firewalls permiten construir políticas de seguridad más o menos estrictas según las necesidades de la red, balanceando la protección con la eficiencia del tráfico.

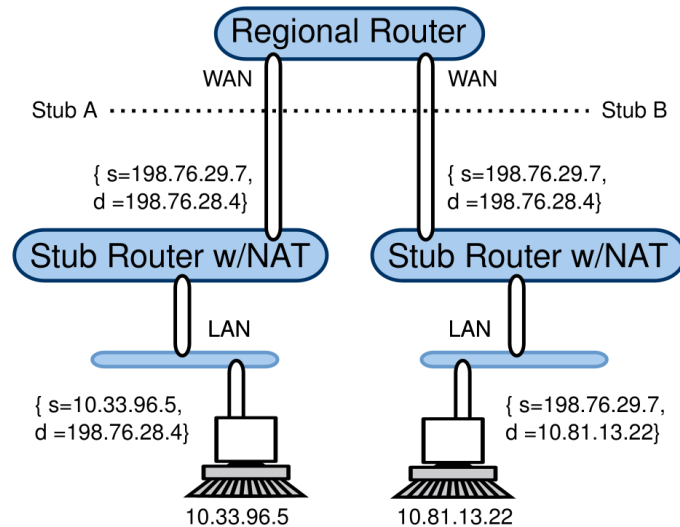
9 NAT

NAT (Network Address Translation) es un método para mapear direcciones IP de un dominio (usualmente privado) a otro (usualmente público). Su forma más básica está especificada en el [RFC 1631](#). La función de NAT se implementa en los gateways y es transparente para los hosts, habiéndose ideado principalmente para mitigar el problema de la escasez de direcciones IPv4. Esto permite que pools de direcciones privadas puedan reutilizarse arbitrariamente sin conflictos con la red pública.

Detalles de NAT

- NAT no solo modifica la dirección IP de los paquetes, sino que también debe actualizar el **checksum** correspondiente.
- Esto incluye el *checksum* de nivel de transporte, el cual depende del *pseudoheader* que incluye la IP.
- En general, cualquier campo donde aparezca la IP en el paquete debe ajustarse; por ejemplo, en paquetes ICMP con payload.
- Potenciales problemas de NAT incluyen:
 - Aplicaciones que pueden dejar de funcionar correctamente.
 - Consideraciones de seguridad al exponer servicios internos mediante NAT.

9.1 NAT ilustrado



Tenemos dos redes locales (LAN) con hosts internos que necesitan comunicarse hacia el exterior a través de un router con NAT. Cada LAN tiene un stub router que realiza NAT para sus hosts. La WAN es la red pública a la que acceden los hosts internos. Las direcciones IP privadas son típicamente del rango 10.x.x.x, mientras que las públicas pertenecen al rango global, por ejemplo 198.76.x.x.

Host interno Stub A:

- IP privada: 10.33.96.5
- Desea enviar un paquete a un destino público: 198.76.28.4
- El paquete original tiene:
 - Dirección de origen: 10.33.96.5
 - Dirección de destino: 198.76.28.4

Host interno Stub B:

- IP privada: 10.81.13.22
- Desea enviar un paquete al mismo destino público: 198.76.28.4
- El paquete original tiene:
 - Dirección de origen: 10.81.13.22
 - Dirección de destino: 198.76.28.4

Funcionamiento del NAT

- El router de Stub A realiza NAT y reemplaza la IP privada de origen por la IP pública del router: 198.76.29.7.
- El router de Stub B realiza NAT usando la misma IP pública 198.76.29.7, pero asignando un puerto distinto para diferenciar las sesiones (NAPT).
- Ambos hosts internos pueden enviar paquetes hacia la WAN usando la misma IP pública, manteniendo su destino original.
- La traducción de direcciones es transparente para los hosts internos y permite reutilizar la IP pública para múltiples sesiones simultáneas.

9.2 NAPT (Network Address and Port Translation)

NAPT es una extensión de NAT que permite el mapeo de muchos-a-uno, a diferencia del mapeo biyectivo de NAT tradicional. La dirección pública es compartida por todos los hosts de la red privada, mientras que los puertos públicos son los que identifican el tráfico y definen el mapeo. Esta terminología está descrita en el [RFC 2663](#).

Funcionamiento de NAPT

- Cuando un host de la red privada quiere contactar un servicio externo, el paquete saliente se modifica para incluir un **puerto origen distinto** que identifica la sesión.
- El gateway realiza esta modificación y guarda la entrada en su **tabla de mapeos**, lo que permite que las respuestas puedan ser correctamente redirigidas al host interno adecuado.
- Un host externo no puede iniciar comunicación directamente con la red privada a menos que exista una regla explícita de **port forwarding**.
- Ejemplo de port forwarding: $(190.172.12.16, 8080) \rightarrow (192.168.0.100, 80)$, donde el gateway redirige cualquier conexión al puerto público 8080 hacia el puerto 80 del host interno.